MODULE 03 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Micro-partitions, pruning and clustering

The storage internals that most performance questions quietly resolve to.

DOMAIN 1 of 5 WEIGHT 31% MODULE 3 of 31

admin@sthithapragna.com

Snowflake has no indexes to create. Query speed comes from **not reading data**, and the layout decides how much you skip.

What a micro-partition is

The numbers here are quoted directly in questions, so learn them as written

- **50 MB to 500 MB of uncompressed data.** The stored size is smaller, because data is always stored compressed.
- A **contiguous unit of storage** holding a group of rows, organised **columnar** within the partition.
- Created **automatically on every Snowflake table**. You never define or maintain them.
- Partitioned **using the order the data arrives in** as it is inserted or loaded.
- **Immutable.** DML never edits one. It writes new micro-partitions and marks the old ones as no longer current.

> ON THE EXAM
>
> A large table can hold millions, even hundreds of millions of micro-partitions. That granularity is the point.

Why the design works

The documented benefits, and each one is a possible answer

- **Derived automatically.** No static partitioning scheme to define up front or maintain.
- **Small.** That enables efficient DML and fine-grained pruning.
- **They may overlap in value range.** Combined with uniform small size, that is what *prevents skew*.
- **Columnar.** Only the columns a query references are scanned.
- **Compressed per column, per partition**, with Snowflake choosing the algorithm each time.

> ON THE EXAM
>
> Overlap is listed as a benefit, not a defect. Traditional static partitioning cannot overlap, and that is exactly why it skews.

The metadata that makes it all work

SNOWFLAKE RECORDS	USED FOR
The range of values for each column	Deciding a partition cannot contain a match, so it can be skipped.
The number of distinct values	Cost estimation and plan choice by the optimiser.
Additional properties	Optimisation and efficient query processing.

ON THE EXAM

This lives in the cloud services layer, which is why COUNT(*) and MIN/MAX can answer with no warehouse running.

How pruning actually runs

01

Read metadata

Compare the query's filter against each partition's value ranges.

02

Prune partitions

Skip every micro-partition that cannot contain a match.

03

Prune columns

Within survivors, read only the columns the query references.

04

Scan what is left

Whatever remains is the only data the warehouse touches.

ON THE EXAM

Two independent savings. Fewer partitions, and fewer columns within them. A SELECT * gives up the second one.

How good is the pruning, and when does it fail

- **The documented ideal:** a filter selecting 10% of a range should scan about **10% of the micro-partitions**.
- The closer the **ratio of partitions scanned to data actually selected**, the better the pruning.
- Pruning works on **semi-structured columns** too, not just ordinary ones.
- **Snowflake does not prune on a predicate containing a subquery**, even if that subquery returns a constant.
- Wrapping the filter column in a **function** generally defeats pruning. Filter on the raw column where you can.

> ON THE EXAM
>
> Query Profile reports partitions scanned against partitions total. That ratio is the diagnosis.

Two consequences of immutability

- **Some operations are metadata-only.** Deleting every row from a table does not rewrite anything.
- **Dropping a column does not rewrite the micro-partitions.** The dropped column's data stays in storage.
- Old versions of partitions are what **Time Travel** reads. Nothing was overwritten, so the past is still there.
- A **clone** is a new set of pointers to existing partitions. Immutability is why that is safe.

> ON THE EXAM
>
> If you drop a column expecting the storage bill to fall, it will not. That surprises people, and it is documented behaviour.

Pruning only helps if the rows you want are **together**. Which brings us to clustering.

Natural clustering, and how it decays

- **Every table is clustered by something**, whether you asked for it or not: the order the data was loaded in.
- Time-series data loaded chronologically is **naturally well clustered on time**. Often nothing more is needed.
- **DML degrades it.** INSERT, UPDATE, DELETE, MERGE and COPY gradually scatter related rows across partitions.
- Loading in an order unrelated to how you query means **poor clustering from day one**.

> ON THE EXAM
>
> Snowflake does not recluster a table with no explicit CLUSTER BY. Natural clustering decays and stays decayed.

Measuring it: overlap and depth

- Snowflake tracks the **total partitions**, how many **overlap** in value range, and the **depth** of that overlap.
- **Clustering depth** is the average depth of overlapping partitions. It is 1 or greater, and **smaller is better**.
- An **empty table has a depth of 0**. A table with no overlap at all is in a **constant state**.
- Depth is **not an absolute measure**. The documentation says query performance is the real indicator.

> ON THE EXAM
>
> Reaching a constant state on a real table is described as 'neither likely nor required'. Do not chase depth 1.

The two functions to know by name

FUNCTION	WHAT IT GIVES YOU
SYSTEM$CLUSTERING_DEPTH	The average clustering depth for the specified columns. A single number.
SYSTEM$CLUSTERING_INFORMATION	Depth plus the fuller picture: partition counts, overlap histogram, and which clustering version the table uses.
AUTOMATIC_CLUSTERING_HISTORY	Account Usage view. Credits consumed reclustering, per table, over time.

ON THE EXAM

Learn the two SYSTEM$ names as written. Questions quote them exactly and the distractors are near-misses.

When to define a clustering key

- The table is **large**. Snowflake's guidance points at tables in the terabyte range, with many micro-partitions.
- Queries **filter or join selectively** on the same column or two, repeatedly.
- Query performance has **degraded over time** as DML accumulated.
- **Not** for small tables. Reclustering cost will exceed any benefit.
- **Not** for tables written far more often than they are read. Every write makes more work.

> ON THE EXAM
>
> Clustering is not free and not automatic-by-default. It is an opt-in trade of ongoing credits for faster reads.

Choosing the key

- **Order from lowest to highest cardinality.** A high-cardinality column first ruins clustering on the ones after it.
- **Cardinality in the middle.** High enough to prune well, low enough to group rows together.
- Prefer columns used in **selective filters**, then **join predicates**. Favour those over ORDER BY or GROUP BY columns.
- Snowflake recommends **no more than three or four columns**. Each one adds reclustering cost.
- For a very high-cardinality column, cluster on an **expression** that coarsens it, such as a truncated date.

> ON THE EXAM
>
> A unique identifier is the classic bad key. Perfect cardinality, ruinous maintenance cost, and it prunes nothing useful.

Clustering key or search optimization?

CLUSTERING KEY

Changes where the data sits

- Reorganises the physical layout
- Ranges and equality: BETWEEN, >=, =
- Medium-cardinality columns
- 'All events for tenant 42 last week'

SEARCH OPTIMIZATION SERVICE

Adds a separate access path

- Builds a search access path alongside
- Point lookups, IN lists, VARIANT, geospatial
- High-cardinality columns
- 'The one event with this exact id'

THE TIEBREAKER

They are not substitutes and can be used together. Range scan, cluster. Needle in a haystack, search optimization.

What Automatic Clustering costs, and how to stop it

- Runs in the **background with no warehouse** of yours, and bills as **serverless** credits.
- **Non-blocking.** DML against the table continues while reclustering runs.
- Reclustering deletes and re-inserts rows into new partitions, so it costs **storage as well as credits**.
- **ALTER TABLE … SUSPEND RECLUSTER** stops it, and stops the charges, without dropping the key.
- Dropping the clustering key prevents all future reclustering permanently.

> ON THE EXAM
>
> Reclustering does not begin the moment you define a key. Snowflake reclusters only if the table will benefit.

New: Optima Clustering, from 1 September 2026

- Automatic Clustering now has two versions. **Newly clustered tables use Optima Clustering** from that date.
- Tables clustered **before** then stay on **Clustering Classic**. There is no forced migration.
- **Classic uses only the first 5 bytes** of each clustering key column. Optima allows up to 1 KB across all columns.
- Optima bills on **data volume ingested** rather than compute hours, which makes the cost predictable.
- This landed days before this course was recorded. **Answer exam questions from the Classic behaviour.**

> ON THE EXAM
>
> The 5-byte limit is why the docs suggest clustering on a substring when a string column's leading characters are identical.

A 6 TB events table is queried many times a day with filters on event_date, usually for a single week. The table is loaded continuously throughout the day from several sources, and rows arrive out of order.

The Query Profile for a typical query shows partitions scanned of 41,900 out of 42,300 total. Bytes spilled to local storage is zero. The warehouse is not queuing.

What is the most effective change?

A Increase the warehouse from Large to 3X-Large so the scan completes faster.

B Define a clustering key on event_date so related rows are grouped into fewer micro-partitions.

C Enable the Search Optimization Service on the table.

D Create a materialized view over the table filtered to the last seven days.

◆ PAUSE AND DECIDE

What is the most effective change?

A. Increase the warehouse to 3X-Large.

Zero spilling and no queuing means the warehouse is not the constraint. This buys speed by paying eight times more to read data you should be skipping.

B. Define a clustering key on event_date.

Scanning 99% of partitions to return one week is a pruning failure, and out-of-order loading is the documented cause.

C. Enable the Search Optimization Service.

Built for point lookups on high-cardinality columns. This is a range scan on a date.

D. A materialized view for the last seven days.

'Usually a week' is not always the same week. The view would miss any query outside its filter.

THE SIGNAL

Almost 42,000 partitions scanned of 42,300, to return one week of a multi-year table. That single ratio is the diagnosis.

> A team read that clustering improves performance and applied clustering keys across their warehouse, including a 3 GB reference table holding product categories that is joined into most queries.
>
> Two weeks later, the Automatic Clustering line on the bill has grown noticeably. Query times on the reference table are unchanged.

What is happening, and what should they do?

A The key is on the wrong column. Recreate it on the join column instead.

B Suspend Automatic Clustering account-wide, and recluster manually each month.

C Reclustering has not finished. Wait for it to reach a constant state.

D The table is too small to benefit. Drop the clustering key from it.

◆ PAUSE AND DECIDE

What is happening, and what should they do?

A Wrong column. Recreate on the join column.

Assumes clustering was the right tool. A 3 GB table has too few partitions for any key to help.

B Suspend account-wide and recluster manually.

Manual reclustering is deprecated, and this would also disable clustering on the large tables where it is working.

C Wait for a constant state.

The docs call reaching a constant state neither likely nor required. Waiting means paying indefinitely.

D Too small to benefit. Drop the key.

Clustering pays off on large tables with many partitions. Dropping the key stops all future reclustering charges.

THE SIGNAL

The right answer here is to stop doing something. 'Apply the good practice everywhere' is itself the mistake.

> An engineer is defining a clustering key for a large multi-tenant events table. Queries almost always filter on both event_date and tenant_id together. There are around 900 tenants and four years of daily data.
>
> The engineer's first draft is CLUSTER BY (event_id, tenant_id, event_date, region, source_system), where event_id is a unique identifier.

What is the main problem with this key?

A. A unique column leads, and the columns are not ordered from lowest to highest cardinality.

B. Nothing. More columns give the optimiser more opportunities to prune.

C. Clustering keys cannot contain more than two columns, so the statement will fail.

D. event_date should be removed, because dates are already handled by natural clustering.

◆ PAUSE AND DECIDE

What is the main problem with this key?

A A unique column leads, and the order is wrong.

A unique leading column is the worst case. It also destroys clustering on every column after it.

B Nothing. More columns prune more.

Backwards. Each extra column adds reclustering cost, and Snowflake recommends no more than three or four.

C Keys are limited to two columns.

There is no such limit. Three to four is a recommendation, not a hard cap.

D Remove event_date, natural clustering handles it.

Natural clustering is load order, which decays. event_date is the best candidate for the leading position.

THE SIGNAL

Better: CLUSTER BY (event_date, tenant_id). Coarsest first, two columns, and both appear in the actual filters.

> A table is clustered on region, a column holding values such as 'EMEA' and 'APAC'. Queries against it were fast for months.
>
> A reporting team has rewritten its dashboard queries to filter with WHERE UPPER(region) = 'EMEA', because some newer rows arrived in mixed case. Since then the dashboards scan almost the whole table, and the clustering depth reported for the table has not changed.

Why did pruning stop working?

A The clustering key was invalidated by the mixed-case rows and must be redefined.

B Automatic Clustering suspended itself because the table reached a constant state.

C The function on the filter column defeats pruning, because the stored metadata describes the raw values.

D String columns cannot be clustered, so the key was never having any effect.

◆ PAUSE AND DECIDE

Why did pruning stop working?

A The key was invalidated by mixed-case rows.

Clustering depth is unchanged, which the scenario states. The physical layout is fine; the query changed.

B Automatic Clustering suspended at a constant state.

Reaching a constant state does not suspend anything, and it would not slow queries if it did.

C The function defeats pruning. Metadata describes raw values.

Ranges are recorded for the stored column. Snowflake cannot tell which partitions satisfy a predicate on the function's output.

D String columns cannot be clustered.

They can. Classic uses only the first 5 bytes of each column, which is a different limitation entirely.

THE SIGNAL

The clue is that clustering depth did not change. The data is laid out exactly as before. Only the query stopped being able to use it.

What to carry into the next module

- **50 to 500 MB uncompressed**, columnar, immutable, automatic. Metadata: value ranges, distinct counts, more.
- **Pruning is two-stage:** partitions first, then columns. Subqueries and functions in a predicate defeat it.
- **Natural clustering is load order**, and it decays under DML. Snowflake will not fix it unless you define a key.
- **Depth: smaller is better, 1 or more, 0 for an empty table.** Not an absolute measure.
- **Do not cluster small or write-heavy tables.** Lowest cardinality first, three or four columns at most.

> ON THE EXAM
>
> Next: table types and view types. Permanent, transient, temporary, external, Iceberg, dynamic, hybrid, and the three kinds of view.